agentcontainer Whitepaper

Abstract

agentcontainer is a transport and execution substrate for mobile software agents represented as authenticated Python source files. Each agent can be deployed as live code into a remote TCP container, can later clone or move across a federated tree of containers, and can report back to the local stage from which it was launched. The container provides operating primitives, not an opinionated LLM backend. This decoupling allows agents to remain autonomous in their choice of reasoning engines and external APIs.

1. Problem Statement

Modern AI systems are usually centralized around a fixed inference endpoint and a server-side orchestration layer. That model is effective for many SaaS architectures, but it is less effective for environments where:

- â¢ data is physically distributed across departments, labs, or edge nodes;
- â¢ movement of lightweight logic is cheaper than movement of entire datasets;
- â¢ agents need local system access near the data they inspect;
- â¢ operators want a neutral substrate instead of a monolithic "AI platform".

agentcontainer addresses this by defining a node that accepts authenticated Python agents as text, loads them dynamically, and gives them a set of explicit host primitives for file access, process execution, HTTP egress, introspection, and mobility.

2. Design Principles

The system follows five principles.

First, every message is explicitly authenticated. Authentication is part of the application payload model, not an external session.

Second, the agent is the portable unit. The source file is not merely a script uploaded

to a server; it is the identity-bearing artifact that can survive relocation.

Third, the container is intentionally neutral regarding LLM providers. Agents may call external reasoning services, but the container itself does not own the model contract.

Fourth, federation is hierarchical. A tree is operationally simpler than a full mesh and maps well to organizations such as departments, labs, and workstations.

Fifth, a launch stage is a first-class concept in the operator workflow. An agent can start from a local stage, travel to a destination container, and return to the stage with a result or inspection report.

Sixth, the first implementation should optimize for conceptual clarity. Stronger isolation, policy, and cryptographic identity can be layered later.

3. System Model

An agentcontainer node exposes a TCP port. Clients and agents send JSON Lines messages. Each message contains a timestamp, a nonce, a sender identity, a payload, and a signature. Messages are explicitly signed at the application level instead of relying on a transport session.

The node maintains an in-memory registry of live agents. An agent source file defines at minimum:

- â¢ AGENT_ID
- â¢ AGENT_SECRET
- â¢ Agent class

The Agent class may implement an activation hook and a message handler. Once loaded, the agent receives a runtime context object that exposes host primitives such as file search, file reads, process execution, HTTP requests, mobility, stage awareness, and runtime capability inspection.

4. Mobility Semantics

Mobility is expressed by two primitives.

Clone creates a new live copy of the current agent on another node while preserving the

current local instance.

Move serializes the current source code and deploys it to another node, then removes the local instance after successful activation on the destination.

In both cases, the agent source is re-sent as text over the protocol. The destination node does not need shared state with the source node beyond the authenticated message it receives.

In the current CLI workflow, an operator may also launch an agent from a local stage. The stage is a temporary local container that sends the agent to the target and waits for it to return. This makes agent travel observable and easy to test without requiring a permanently running orchestration layer.

5. Federation

Federation is represented as a tree. Each node knows its children through static configuration. The runtime exposes a networks primitive so that an agent can inspect the currently configured topology and decide how to traverse it. A tree structure is sufficient for many enterprise or laboratory deployments where containers are arranged by office, team, department, or rack. A stage may appear to the agent as the root of a temporary local tree that includes the immediate target container.

6. Reasoning and External LLMs

The system deliberately separates orchestration from inference. agentcontainer is not an LLM gateway. Instead, an agent can hold its own API secret and directly call an external reasoning service through the host HTTP primitive or its own imported libraries. This separation prevents the container from becoming a bottleneck for model selection, key management, or provider-specific assumptions.

7. Security Discussion

The implemented base offers message integrity and straightforward authentication, but it does not claim hardened isolation. Dynamic Python execution is inherently powerful and therefore dangerous if untrusted code can enter the system. The current design should be treated as a trusted-environment prototype suitable for labs, internal networks, or controlled experiments.

The trust bootstrap for a newly arriving agent is intentionally simple: the message is validated using the secret transported with the agent. This is enough for a first demonstrator but not enough for hostile networks. A production system should move toward public-key identities, signed artifacts, policy engines, and sandboxed execution.

8. Example Deployment

Consider a laboratory with one root node and two departmental nodes. Each node exposes local documents. A travelling scout agent is deployed on the root node with the query "chess". It searches locally, inspects the federation tree, clones itself into each child, and collects distributed findings close to where the documents live. A second demonstration agent can be launched from a local stage, inspect the primitives and resources exposed by a destination container, and return automatically to the stage with its report. The same pattern can be generalized to indexing, compliance inspection, software inventory, or environment diagnostics.

9. Implementation in This Repository

The repository includes:

- â¢ a Python asyncio server implementing the TCP protocol;
- â¢ a unified CLI for service startup, local stage launch, sending, and invocation;
- â¢ a dynamic runtime and agent registry;
- â¢ a mobile example agent catalog;
- â¢ Docker Compose topology for multi-node testing;
- â¢ automated tests for the core flows.

10. Future Work

The natural roadmap includes:

- â¢ capability-based security for primitives;
- â¢ per-agent sandboxing via subprocesses or microVMs;
- â¢ stronger identity and artifact signatures;
- â¢ persistent agent state and event logs;
- â¢ discovery and governance for larger federations;
- â¢ richer result routing and aggregation primitives;

â¢ more explicit public/private key distribution for trusted container operators.

Conclusion

agentcontainer proposes a simple but expressive model: move authenticated executable agents to the data, provide them with explicit operational primitives, and leave reasoning-provider choice to the agent itself. With the addition of stage-based workflows, the system also gives operators a practical way to launch, observe, and receive mobile agents during experiments. The result is a compact substrate for work on mobile AI systems, edge automation, and distributed agent orchestration.